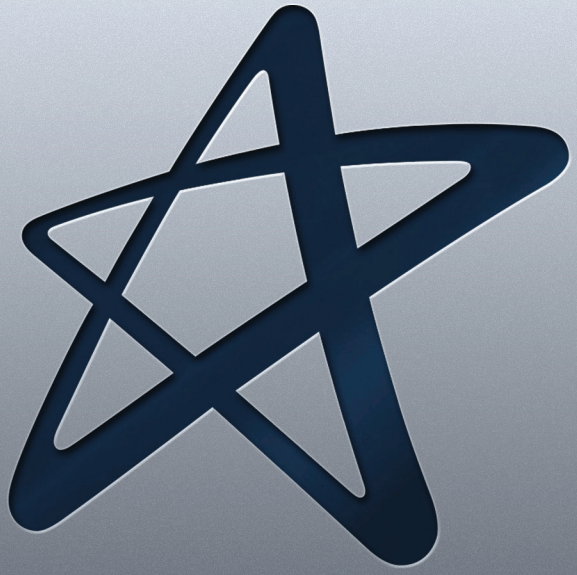
Programação para Dispositivos Móveis



Cruzeiro do Sul Virtual
Educação a Distância

Material Teórico

Utilização dos recursos: Ligação, SMS, navegação web, GPS e envio de e-mail

Responsável pelo Conteúdo:
Prof. Ms. Luis Carlos Reis

Revisão Textual:
Prof. Ms. Douglas Almendro

Revisão Textual:
Profa. Ms. Luciene Oliveira da Costa Santos

UNIDADE Utilização dos recursos: Ligação, SMS, navegação web, GPS e envio de e-mail

- Introdução
- Permissões
- Efetuando ligações
- Enviando SMS
- Navegando na Web
- Utilização do GPS
- Envio de email



OBJETIVO DE APRENDIZADO

- Demonstrar como os dispositivos móveis podem estar conectados por meio de aplicações a internet como SMS, navegação web, GPS e envio de e-mail.

ORIENTAÇÕES

Caro(a) aluno(a),

Estamos começando nossos estudos sobre desenvolvimento mobile, mais especificamente estamos falando de Android.

Com o conhecimento adquirido com os conceitos iniciais, iremos trabalhar com solicitações, Ligação, SMS, navegação web, GPS e envio de e-mail.

Para obter um bom aproveitamento nesta unidade, vamos conferir sua estrutura:

Conteúdo Teórico: neste link, você encontrará o material principal de estudos na forma de texto escrito.

Atividade de Sistematização: os exercícios disponibilizados são de autocorreção e visam que você pratique o que aprendeu na disciplina e que identifique os pontos em que precisa prestar mais atenção, ou pedir esclarecimentos a seu tutor. Além disso, as notas atribuídas aos exercícios serão parte de sua média final na disciplina.

Atividade de Aprofundamento: ver a atividade que será disponibilizada.

Material Complementar e Referências Bibliográficas: nestes links, você poderá ampliar seus conhecimentos.

Lembre-se da importância de realizar todas as atividades propostas dentro do prazo estabelecido para cada unidade. Dessa forma, você evitará que o conteúdo se acumule e que você tenha problemas ao final do semestre.

Uma última recomendação: caso tenha problemas para acessar algum item da disciplina, ou dúvidas com relação ao conteúdo, entre em contato com seu professor-tutor por meio do botão mensagens.

Bom estudo!

Contextualização

Uma das principais funcionalidades dos sistemas móveis é a possibilidade de mantê-los conectados em rede. Por isso devemos explorar melhor as práticas de conectar os aplicativos na internet.

Hoje em dia, os dispositivos possuem ótimos recursos para conexões de rede sem fio e redes móveis e, muitas vezes, as aplicações necessitam desses recursos para acessar e fornecer dados e serviços. O Android possui uma série de classes que permitem essa programação por meio de Socket, URL, Java.net, HttpRequest e HttpResponse.

Bom estudo!

Introdução

Nesta aula, iremos utilizar alguns recursos que a maioria dos celulares possui:

1. Ligação;
2. SMS;
3. Navegação Web;
4. GPS;
5. Envio de Email.

Aproveitaremos o exemplo que finalizamos na aula III, e iremos incrementá-lo com os recursos acima descritos.

Permissões

Vale lembrar que, para utilizarmos alguns recursos do celular, será necessário alterar o arquivo **AndroidManifest.xml**. Assim, quando a aplicação for instalada em um Android, essas permissões serão listadas para que o usuário autorize sua utilização. O Android controla o que as aplicações podem acessar através de permissões.

```
<uses-permission android:name="android.permission.ACCESS_
FINE_LOCATION"/>
```

Permite usar provedores de localização;

```
<uses-permission android:name="android.permission.ACCESS_LO-
CATION_EXTA_COMMANDS"/>
```

Permite acessar comandos adicionais de localização;

```
<uses-permission android:name="android.permission.INTERNET"/>
```

Permite acesso à internet;

```
<uses-permission android:name="android.permission.CALL_PHONE"/>
```

Permite fazer chamadas telefônicas;

```
<uses-permission android:name="android.permission.RECEIVE_SMS"/>
```

Permite receber SMS;

```
<uses-permission android:name="android.permission.READ_SMS"/>
```

Permite ler SMS;

```
<uses-permission android:name="android.permission.SEND_SMS"/>
```

Permite enviar SMS;

```
<uses-permission android:name="android.permission.WRITE_SMS"/>
```

Permite escrever SMS;

```
<uses-permission android:name="android.permission.WRITE_SMS"/>
```

Permite exibir algo para o usuário;

```
<uses-permission android:name="android.permission.CAMERA"/>
```

Permite acessar a câmera;

```
<uses-permission android:name="android.permission.READ_CON-
TACTS"/>
```

Permite ler os contatos;



A lista completa de permissões pode ser encontrada no endereço:
https://goo.gl/vWolU

Portanto, antes de iniciarmos nossa aula, abra o projeto realizado na aula III e altere o arquivo **AndroidManifest.xml**, conforme descrito abaixo:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="com.example.usuario.cadastroalunos" >

    <uses-permission android:name="android.permission.ACCESS_
FINE_LOCATION"/>
    <uses-permission android:name="android.permission.ACCESS_
LOCATION_EXTA_COMMANDS"/>
    <uses-permission android:name="android.permission.INTERNET"/>
    <uses-permission android:name="android.permission.CALL_
PHONE"/>
    <uses-permission android:name="android.permission.RE-
CEIVE_SMS"/>
    <uses-permission android:name="android.permission.READ_
SMS"/>
    <uses-permission android:name="android.permission.SEND_
SMS"/>
   <uses-permission android:name="android.permission.WRITE_
SMS"/>
   <uses-permission android:name="android.permission.WRITE_
VIEW"/>
```

```
    <uses-permission android:name="android.permission.CAM-
ERA"/>
    <uses-permission android:name="android.permission.READ_
CONTACTS"/>

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">
        <activity
            android:name=".Alunos"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.
MAIN" />

                <category android:name="android.intent.cat-
egory.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity
            android:name=".Formulario"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.
MAIN" />
                <category android:name="android.intent.cat-
egory.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Efetuando ligações

Para efetuar ligações telefônicas com o Android, existem duas maneiras.

A primeira, o Android permite que você acesse em baixo nível o telefone e faça e controle as ligações telefônicas. Só que para isso você precisaria ter um conhecimento profundo de como é o cliclo de vida de uma ligação telefônica.

A segunda, que é mais fácil de utilizar: basta chamar uma intenção (*Intent*) de ligar para alguém. Portanto, iremos trabalhar com essa segunda opção, reaproveitar nosso projeto realizado na aula III e incorporar essa funcionalidade. Relembramos que já temos uma funcionalidade de "Ligar" quando é disparado um clique longo no projeto realizado na aula III.

O Android já possui muitas activities prontas para utilizar os vários recursos do dispositivo. O construtor da nossa Intent receberá uma "String", que representa a ação que desejamos executar, em que diversas dessas strings podem ser acessadas através de constantes declaradas na própria classe *Intent*.

Para efetuarmos a ligação, iremos precisar utilizar a Intent "Intent.ACTION_CALL", já configurada conforme descrito acima no arquivo **AndroidManifest.xml**. A ligação será efetuada para os dados que serão transmitidos para a outra activity, através do método *setData*. A *action* e a data são as principais informações dessa *Intent*.

Portanto, iremos alterar o método *onCreateContextMenu,* modificando o item de menu "Ligar", conforme exemplo abaixo.

```
MenuItem ligar = menu.add(0, 0, 0, "Ligar");
ligar.setOnMenuItemClickListener(new MenuItem.OnMenuItem-
ClickListener() {
    public boolean onMenuItemClick(MenuItem item) {
        Uri uri = Uri.parse("tel:" + alunoSelecionado.getTele-
fone().toString().trim());
        Intent intent = new Intent(Intent.ACTION_CALL, uri);
        startActivity(intent);
        return false;
    }
});
```

O *Uri.parse* tem a função de passar chave/valor para o sistema android por meio da string que é uma chave de identificação para efetuar que tipo de ação será "tel:"+ valor. O objeto uri, que é uma referência, recebe a chave/valor e passa para a o construtor da intent, o objeto uri e a ação ou a intenção que a intent deverá executar. O método *startActivity(intent)* vai iniciar a aplicação que efetuará a ligação.

Lembrando que será solicitado o import do objeto "Uri" acima descrito, portanto, no topo da classe utilize a importação:

```
import android.
net.Uri;
```

Execute o projeto e veja a execução.

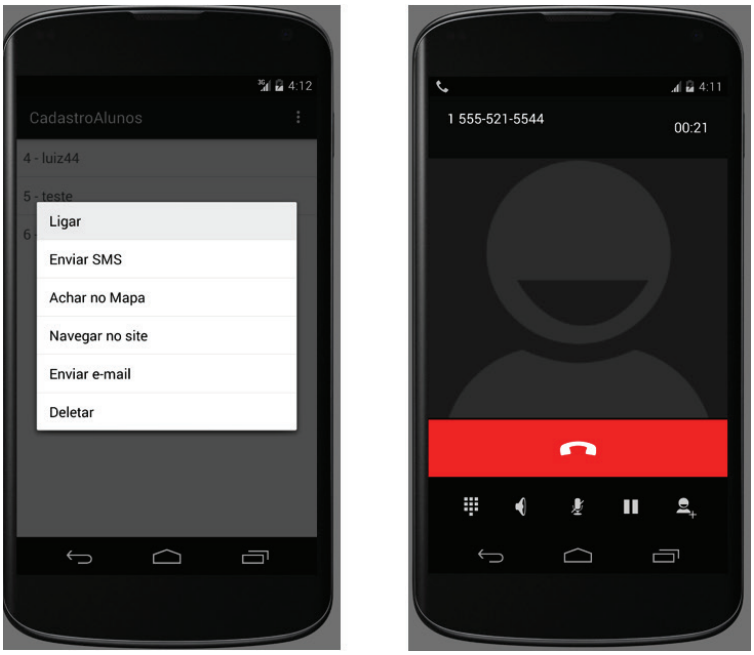


Figura 1

Caso queira chamar o discador somente, você poderá utilizar a Intent "Intent. ACTION_DIAL", permitindo ao usuário discar os números.

Perceba que estamos construindo um Intent diferente do que vimos até agora. Na última aula, deixávamos bastante explícito qual era a activity que estávamos querendo abrir, passando inclusive sua classe, exemplo: "*new Intent(Alunos.this, Formulario.class)*", mas neste exemplo passamos somente uma String no qual nos indicará a nossa intenção, e o Android decidirá, através de uma série de regras que podem ser customizadas, qual *activity* vai ao chamado. Este tipo de intente é implícito, ou seja, não sabemos exatamente quem vai ser iniciado e qual a activity específica, sendo assim, cada dispositivo e aplicação pode configurar, através das configurações avançadas, como cada tipo de Intent deve ser resolvida.

Enviando SMS

O SMS, com significado da sigla, Short Message Service, em português, Serviço de mensagens curtas, é um serviço muito usado para troca de mensagens de textos breves que podem ser enviadas ou recebidas por meio de um aparelho celular.

Adaptaremos nosso exemplo para que seja enviado ao aluno uma mensagem com sua nota. No nosso caso, como não temos cadastrado o e-mail do aluno, iremos deixar fixo um e-mail para que possamos exemplificar nossa aplicação.

Portanto, iremos alterar novamente o método *onCreateContextMenu,* modificando o item de menu "Enviar e-mail", conforme exemplo abaixo:

```
MenuItem sms = menu.add(0,1,0,"Enviar SMS");
sms.setOnMenuItemClickListener(new MenuItem.OnMenuItemClick-
Listener() {
    public boolean onMenuItemClick(MenuItem item) {
        Uri uri = Uri.parse("sms:" + "email_do_aluno@teste.
com.br");
        Intent intent = new Intent(Intent.ACTION_VIEW, uri);
        startActivity(intent);
        return false;
    }
});
```

Utilizamos o método *putExtra* e *setData,* o qual serve para transferir os dados de uma *intent* para a outra.

Execute o projeto e veja a execução.

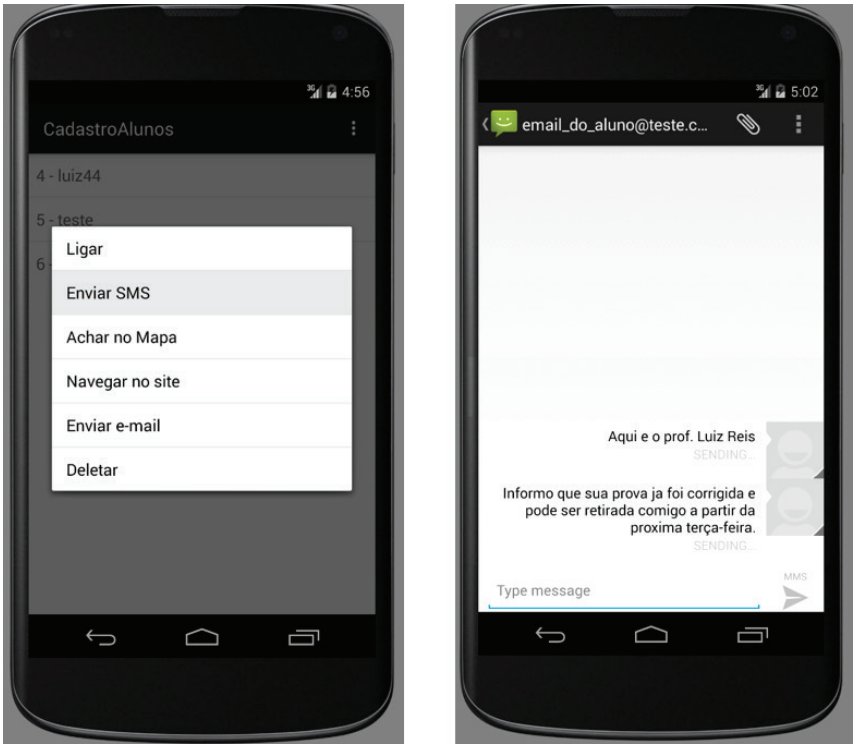


Figura 2

Navegando na Web

Utilizaremos o intent Intent.ACTION_VIEW, que pode abrir o nosso browser, no caso de uma URI que iremos deixar fixo no código.

Portanto, iremos alterar novamente o método *onCreateContextMenu*, modificando o item de menu "Navegar no site", conforme exemplo abaixo:

```
MenuItem web = menu.add(0,3,0,"Navegar no site");
web.setOnMenuItemClickListener(new MenuItem.OnMenuItemClick-
Listener() {
    public boolean onMenuItemClick(MenuItem item) {
        Intent intentWeb = new Intent(Intent.ACTION_VIEW);
        intentWeb.setData(Uri.parse("http://m.uol.com.br"));
        startActivity(intentWeb);
        return false;
    }
});
```

Execute o projeto e veja a execução.

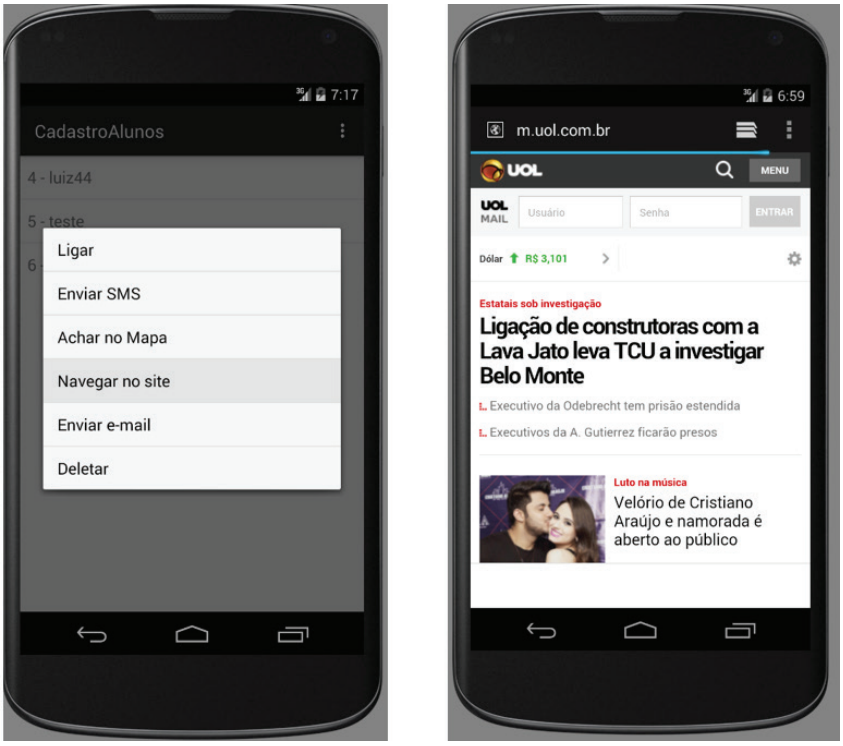


Figura 3

Podemos, também, criar uma nova Activity e um novo layout para que a navegação possa ser executada neste novo layout. Para isso, crie um novo layout na pasta *res\layout* com nome "web.xml".

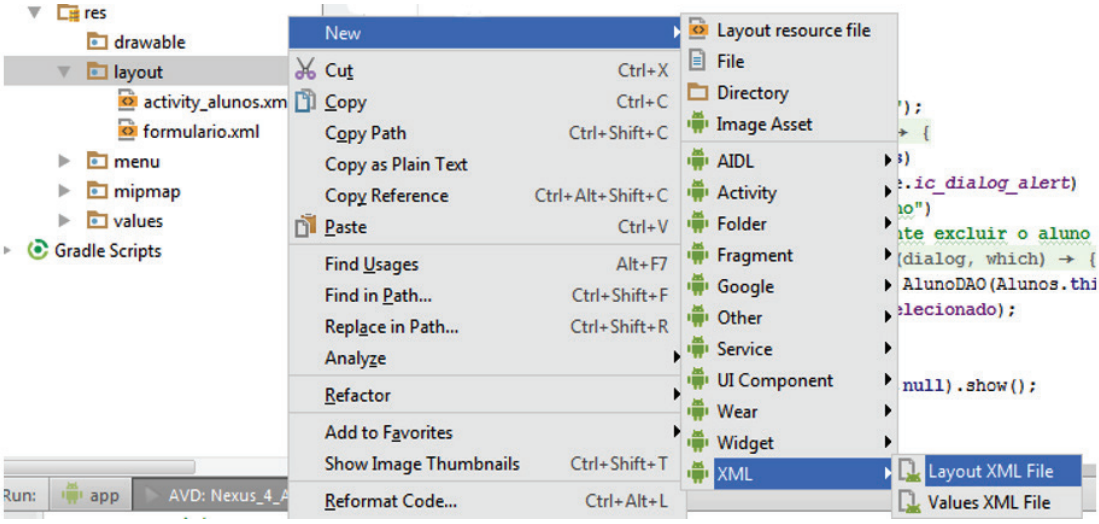


Figura 4

No arquivo web.xml, iremos criar o layout abaixo:

```
<?xml version=”1.0” encoding=”utf-8”?>
<LinearLayout xmlns:android=”http://schemas.android.com/apk/
res/android”
    android:layout_width=”fill_parent”
    android:layout_height=”fill_parent”
    android:orientation=”vertical” >

    <WebView
        android:id=”@+id/webViewSite”
        android:layout_width=”fill_parent”
        android:layout_height=”fill_parent” >
    </WebView>
</LinearLayout>
```

Crie uma nova activity com nome “VisualizarSiteWeb”, que servirá para executá-la após selecionar o item “Navegar no Site”.

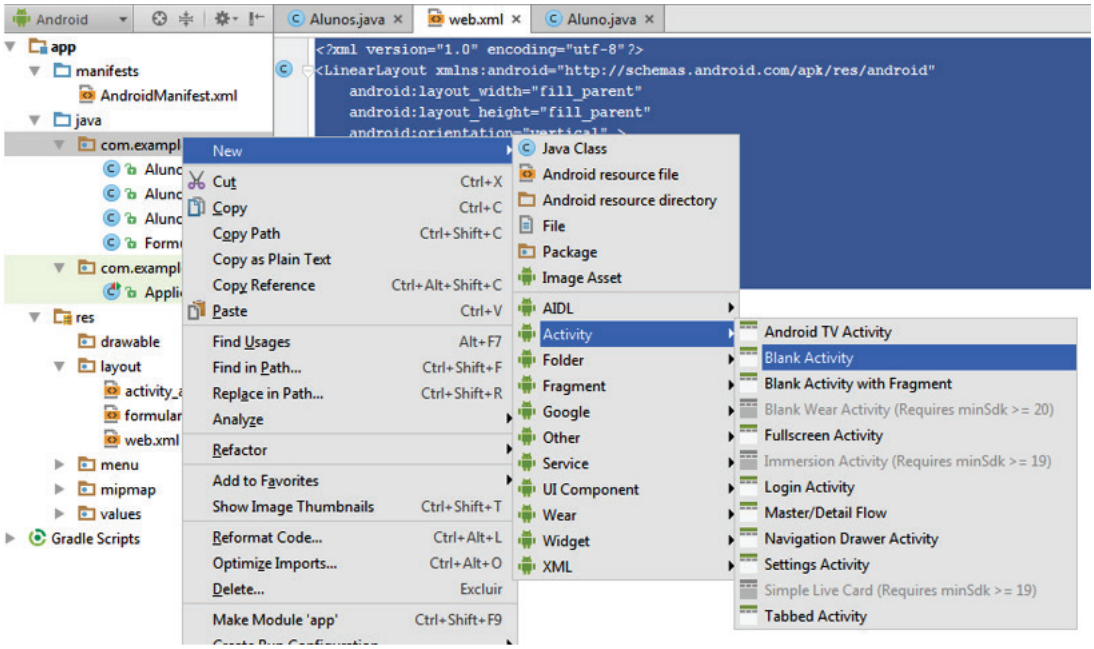


Figura 5

Nesta classe, iremos substituir o código gerado pelo código abaixo:

```
package com.example.usuario.cadastroalunos;

import android.support.v7.app.ActionBarActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.webkit.WebView;
```

```
public class VisualizarSiteWeb extends ActionBarActivity {

    private Aluno aluno;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        setContentView(R.layout.web);

        WebView view = (WebView) findViewById(R.
id.webViewSite);
        aluno = (Aluno) getIntent().getSerializableExtra("alu
noSelecionado");

        String url = "http://m.uol.com.br";
        view.loadUrl(url);
    }
}
```

Alterar novamente o método *onCreateContextMenu* na classe **Alunos.java**.

Alterando o item de menu "Navegar no site", conforme exemplo, substitua pelo código abaixo:

```
MenuItem web = menu.add(0,3,0,"Navegar no site");
web.setOnMenuItemClickListener(new MenuItem.OnMenuItemClick-
Listener() {
    public boolean onMenuItemClick(MenuItem item) {
        Intent edit = new Intent(Alunos.this, VisualizarSite-
Web.class);
        edit.putExtra("alunoSelecionado", alunoSelecionado);
        startActivity(edit);
        return true;
    }
});
```

Agora temos que alterar novamente o arquivo **AndroidManifest.xml**, pois foi criado uma nova activity e devemos mapeá-la dentro deste arquivo.

```
<activity
    android:name=".VisualizarSiteWeb"
    android:label="@string/app_name" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />

        <category android:name="android.intent.category.
LAUNCHER" />
    </intent-filter>
</activity>
```

Execute o projeto e veja a execução.

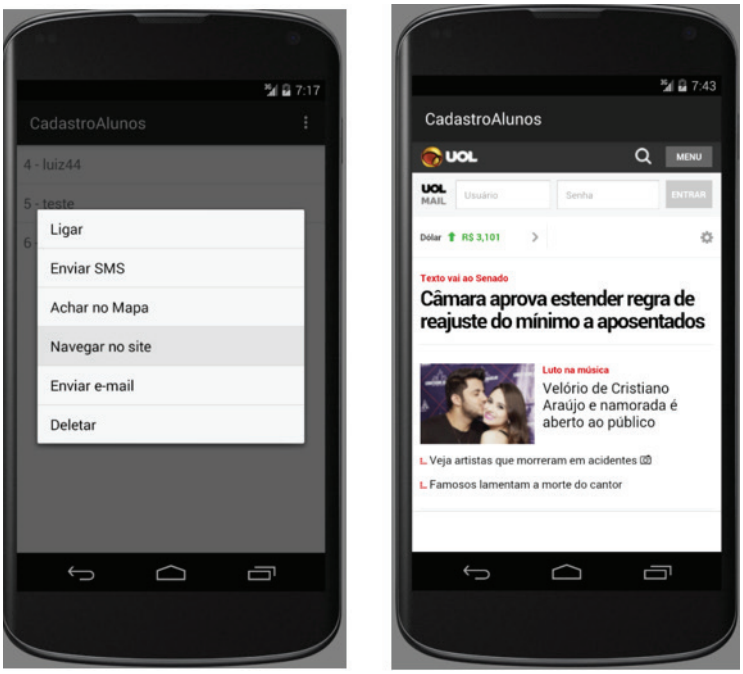


Figura 6

Perceba que agora que não aparece a URI de navegação.

Utilização do GPS

Utilizaremos o intent Intent.ACTION_VIEW para exibir o mapa no Android. Utilizaremos a posição geográfica X, Y como referência e o parâmetro utilizado para isso será o "geo".

Portanto, iremos alterar novamente o método *onCreateContextMenu* alterando o item de menu "Achar no Mapa" conforme exemplo abaixo.

```
MenuItem gps = menu.add(0,2,0,"Achar no Mapa");
gps.setOnMenuItemClickListener(new MenuItem.OnMenuItemClick-
Listener() {
    public boolean onMenuItemClick(MenuItem item) {
        double latitude = 40.714728;
        double longitude = -73.998672;
        String label = "ABC Label";
        String uriBegin = "geo:" + latitude + "," + longitude;
        String query = latitude + "," + longitude + "(" + la-
bel + ")";
        String encodedQuery = Uri.encode(query);
        String uriString = uriBegin + "?q=" + encodedQuery +
"&z=16";
        Uri uri = Uri.parse(uriString);
       Intent intent = new Intent(android.content.Intent.AC-
TION_VIEW, uri);
```

```
            startActivity(intent);
            return true;
        }
    });
```

Envio de email

Utilizaremos o intent Intent.ACTION_VIEW para envio de email com uma série de parâmetros e opcionais.

Portanto, iremos alterar novamente o método *onCreateContextMenu*, modificando o item de menu "Enviara e-mail", conforme exemplo abaixo.

```
MenuItem email = menu.add(0,5,0,"Enviar e-mail");
email.setOnMenuItemClickListener(new MenuItem.OnMenuItem-
ClickListener() {
    public boolean onMenuItemClick(MenuItem item) {
        Intent mail = new Intent(Intent.ACTION_SEND);
        mail.setType("message/rfc822");
        mail.putExtra(Intent.EXTRA_EMAIL, "");
        mail.putExtra(Intent.EXTRA_SUBJECT, "Assunto");
        mail.putExtra(Intent.EXTRA_TEXT, "Texto do email" );

        startActivity(mail);
        return true;
    }
});
```

Execute o projeto e veja a execução.

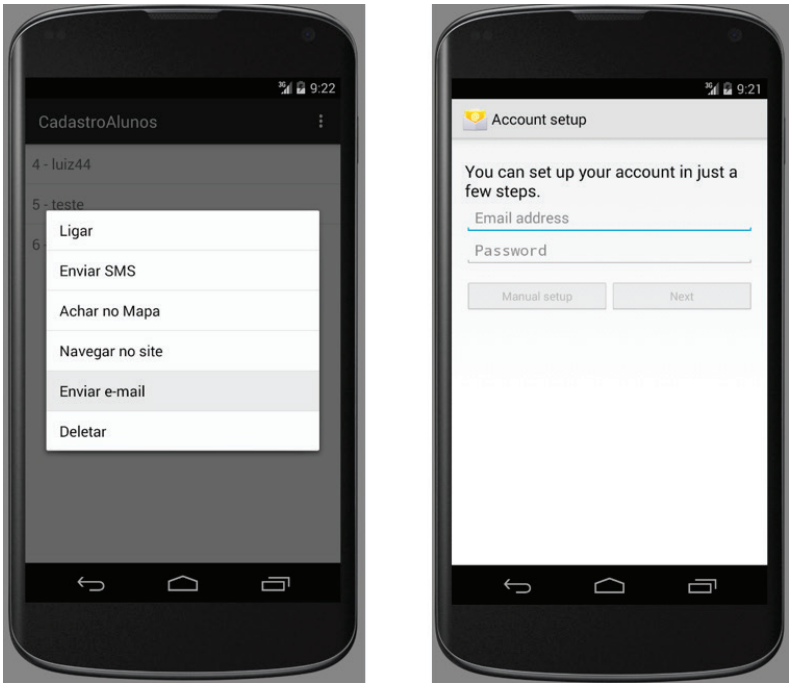


Figura 7

Particularmente, neste caso, irá abrir uma janela para configuração da conta de e-mail.

Após configurar a conta de e-mail, será exibida a tela para envio de e-mail:



Figura 8

Material Complementar

Indicações para saber mais sobre os assuntos abordados nesta Unidade:

Para aprofundar seus estudos sobre threads, verifique os vídeos com um exemplo simples de utilização de thread:

Vídeos

Android enviar email
https://youtu.be/jMYMm3Mcye8

Programación Android - obtener las Coordenadas GPS (Tutorial)
https://youtu.be/DsBTQ4F6n7s

Desarrollo en Android · Localización GPS y Progress Dialog
https://youtu.be/PNsNqHaQP4I

Referências

ANDROID. **Android Developers**. Disponível em http://developer.android.com

LECHETA, R. R. **Google Android, Aprenda a criar aplicações para dispositivos móveis com o Android SDK**. São Paulo: Novatec, 2010.

MEDNIEKS, Z. et. al. **Desenvolvimento de Aplicações Android**. São Paulo: Novatec, 2009.

LEE, W. M. **Introdução ao Desenvolvimento de Aplicativos para o Android**. Rio de Janeiro: Editora Ciência Moderna, 2011

SQLITE. **SQLite**. Site oficial em www.sqlite.org.

Cruzeiro do Sul Virtual
Educação a Distância

www.cruzeirodosulvirtual.com.br
Campus Liberdade
Rua Galvão Bueno, 868
CEP 01506-000
São Paulo - SP - Brasil
Tel: (55 11) 3385-3000


Cruzeiro do Sul
Educacional